

Documentation for raw file read

Thamine Dalichaouch

1 Features

- **Particle Initialization:** Reads in particles once on node 0 and disperses them via mpi to other nodes. Particles are read in chunks to avoid memory bottlenecks. Supports 1D-3D cartesian, 2D-cylindrical, and quasi-3D code.
- **Field Initialization:** Generates the EM fields by accelerating the particles/beams to their final momenta (p1) defined in their RAW file. This feature can be toggled on or off (i.e., defaults to ufl if specified) in the input deck and can be used to study complex beams with arbitrary position x, momentum p, and charge q generated by the user (i.e. using an external python script) or by osiris raw diagnostics. For example, this feature can be used to generate fields from custom 6D phase space distributions of beams that drive potential FACET II experiments.
- **Simulation scaling:** Allows for adjusting x1 particle placement in the simulation box and scaling of macroparticle charge q when changing resolutions. Also accounts for different simulation box boundaries by automatically ignoring particles which are outside the box dimensions.

2 Setup and Testing

- **Setup:** To use initialize (x,p,q) from a raw file for a particular species, the parameter **init_type** must be set to **'file'** in the species section. To generate the fields, use the acceleration routine by setting the parameter **n_accelerate** and set **use_particle_uacc** to **'true.'** to accelerate particles to their final p1 specified in the raw file. It is worth noting that the **use_particle_uacc** flag is completely optional and if unspecified (or set to **'false.'**), the code will default to the pre-existing acceleration routines using **n_accelerate** or **n_q_incr**. Specify the path to the RAW data file in profile section using **file_name**. In the profile section, the optional parameter **q_mult** is the charge scaling factor (defaults to 1.0) and **x1_offset** is the translational factor, i.e. $x1 = x1_{raw} - x1_{offset}$. Instead of automatically scaling the charge q, we left it as an input to the user since they may decide to construct their own custom raw file that does not correspond to an original osiris simulation. A snippet from a typical input deck for reading a raw file is shown below. The entire input deck can be found here

(https://github.com/GoLP-IST/osiris/tree/dev/decks/raw_read) under the name os-stdin-rz-raw-read and reads in a raw file generated by os-stdin-rz.

!-----configuration of the bunch-----

```
species
{
    name = "driver",
    num_par_max = 160000,
    rqm=-1.0,
    num_par_x(1:2) = 2, 2,
    push_type = 'simd',
    free_stream = .true.,

    ! flag for reading from file
    init_type = 'file ',
}

udist
{
    n_accelerate = 2048,

    ! use_particle_uacc is new flag for accelerating particles
    ! to their final p1 momenta obtained from raw files
    ! should be used every time the init_type is set to file
    use_particle_uacc = .true.,
}

profile
{
    ! file to read x,p,q from
    file_name = 'RAW-driver-000000.h5',

    ! The charge scaling factor (defaults to 1.0)
    ! If using a different resolution than the original simulation
    ! which generated the RAW file need to scale by the ratio
    ! of the simulation resolutions
    ! q_mult = 1.0,

    ! Simulation box offset (defaults to 0.0)
    ! center driver in simulation box by subtracting x1_offset
    ! from all x1 values from the raw data file
    ! x1_offset = 0.0,
```

}

- **Examples:** More example input decks can be found in `/decks/raw_read` in the pull request (https://github.com/GoLP-IST/osiris/tree/dev/decks/raw_read). Examples are shown using different resolutions (see `os-stdin-rz-raw-4xresolution`) from the original input deck (`os-stdin-rz`) and shifting the beam over into the $t=0$ simulation box (`os-stdin-rz-raw-x1-shift`) when it is not inside the box dimensions.
- **Basic tests:** Tested file reading from single/double precision RAW files to single/double precision simulations in 2D-cylindrical, quasi-3D, and Full-3D and validated results against a fresh copy of the dev branch. Specifically compared fields, raw macroparticles dumps (x,p,q), and currents at various times for simulations using the new code and current dev branch. The errors were within precision of the RAW file dumped.
- **Large 3D data sets:** Tested the code on 3D RAW files with sizes up to 10GBs. For example, it takes about 20 seconds to read and mpisend data for a double precision 1.8GB RAW file for 32 knl nodes on cori. Most of the time is actually spent reading in the particles.

```
Setting up particles
- Initializing file read of RAW-driver-3D-000000.h5 ...
- Reading 33423360 particles in double precision...
- Read in 1785.000 MB @ 129.3033 MB/s
- Mapping particles to nodes...
- Sending particles to nodes...
Setting up nodes
```

- **Asymmetric Quasi-3D beams:** Tested file read on quasi-3D asymmetric beams (using the `aspect_ratio` parameter in the profile section) using $m \leq 4$ modes and compared q3d diagnostics `e2_cyl_m` (modes 0-4), `j1_cyl_m` (modes 0-4), and raw dumps (x,p,q) to a fresh copy of dev. The errors were within double precision.
- **RAW data precision:** Also tested cases where the RAW file does not have the same precision as the simulation. The raw diagnostic precision is not always tied to the simulation precision and can be modified by going into `source/dutil/os-dutil-f.03` and commenting out either `p_single` or `p_double`.
- **Restarts:** Also verified that the raw file read does not trigger after a restart.

3 Notes

- **Field implementation:** The code was largely designed around importing and accelerating beams rather than full plasma wakefields, which is why EM fields were not

imported. EM field initialization from a hdf5 files (e1-e3, b1-b3) can be done completely independently of the raw file initialization implemented here.

- **Memory features:** The code reads in particles from the RAW file in chunks (default is $1e7$ particles at a time). The chunk size can be varied inside spec/psource/os-psource-file.f03 (see parameter **integer :: part_chunk_size**). For large data sets, this will largely preserve the speedup of reading particles on a single node while avoiding any memory issues on node 0.